实验七

代码

parser.l

```
%{
#include <string.h>
#include <stdlib.h>
#include "util.h"
#include "errormsg.h"
#include "parser.tab.h"

int charPos = 1;

void adjust(void) {
    EM_tokPos = charPos;
    charPos += yyleng;
}
%}

delim       [ \t\r]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
int_num     {digit}+
float_num   ({digit}+\.{digit}*)|(\.{digit}+)

/* 注意：这里必须用 %x，而不是 %s */
%x COMMENT

%%

\n              {adjust(); EM_newline();}
{ws}            {adjust();}

 /* 注释 */
"(*"            {adjust(); BEGIN COMMENT;}
<COMMENT>"*)"   {adjust(); BEGIN INITIAL;}
<COMMENT>\n     {adjust(); EM_newline();}
<COMMENT>.      {adjust();}

 /* 关键字必须放在 ID 前面 */
"PROGRAM"       {adjust(); return PROGRAM;}
"VAR"           {adjust(); return VAR;}
"INTEGER"       {adjust(); return INTEGER;}
```

```
"REAL"          {adjust(); return REAL;}
"BEGIN"         {adjust(); return BEGINN;}
"END"           {adjust(); return END;}
"IF"            {adjust(); return IF;}
"THEN"          {adjust(); return THEN;}
"ELSE"          {adjust(); return ELSE;}
"WHILE"         {adjust(); return WHILE;}
"DO"            {adjust(); return DO;}

 /* 运算符和界符 */
":="            {adjust(); return ASSIGN;}
"<>"            {adjust(); return NEQ;}
"<="            {adjust(); return LE;}
">="            {adjust(); return GE;}
":"             {adjust(); return COLON;}
";"             {adjust(); return SEMICOLON;}
","             {adjust(); return COMMA;}
"."             {adjust(); return PERIOD;}
"("             {adjust(); return LPAREN;}
")"             {adjust(); return RPAREN;}
"+"             {adjust(); return PLUS;}
"-"             {adjust(); return MINUS;}
"*"             {adjust(); return TIMES;}
"/"             {adjust(); return DIVIDE;}
"="             {adjust(); return EQ;}
"<"             {adjust(); return LT;}
">"             {adjust(); return GT;}

 /* 浮点数要放在整数前面 */
{float_num}     {adjust(); yylval.fval = atof(yytext); return FLOAT;}
{int_num}       {adjust(); yylval.ival = atoi(yytext); return INT;}
{id}            {adjust(); yylval.sval = String(yytext); return ID;}

.               {adjust(); EM_error(EM_tokPos, "illegal token: %s", yytext);}

%%

void resetLexState() {
    BEGIN INITIAL;
    charPos = 1;
}

int yywrap() {
    charPos = 1;
    return 1;
}
```

parser.y

```
%{
#include <stdio.h>
#include "util.h"
#include "errormsg.h"

int yylex(void);

void yyerror(char *s)
{
    EM_error(EM_tokPos, "%s", s);
}
%}

%union {
    int ival;
    double fval;
    string sval;
}

%token <sval> ID
%token <ival> INT
%token <fval> FLOAT

%token INTEGER REAL

%token COMMA COLON SEMICOLON LPAREN RPAREN PERIOD
%token PROGRAM BEGINN END VAR IF WHILE DO THEN ELSE
%token ASSIGN EQ NEQ LT LE GT GE
%token PLUS MINUS TIMES DIVIDE

%start program

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

%left EQ NEQ LT LE GT GE
%left PLUS MINUS
%left TIMES DIVIDE
%right UMINUS

%debug

%%

program
```

```
46      : PROGRAM ID SEMICOLON vardec BEGINN stmts END PERIOD
47      ;
48
49  vardec
50      : VAR declist
51      ;
52
53  declist
54      : dec
55      | declist dec
56      ;
57
58  dec
59      : idlist COLON type SEMICOLON
60      ;
61
62  idlist
63      : ID
64      | idlist COMMA ID
65      ;
66
67  type
68      : INTEGER
69      | REAL
70      ;
71
72  stmts
73      : /* empty */
74      | stmtlist
75      ;
76
77  stmtlist
78      : stmt
79      | stmtlist SEMICOLON stmt
80      | stmtlist SEMICOLON
81      ;
82
83  stmt
84      : assignment_stmt
85      | if_stmt
86      | while_stmt
87      | compound_stmt
88      ;
89
90  assignment_stmt
91      : ID ASSIGN exp
92      ;
```

```
 93
 94 if_stmt
 95     : IF exp THEN stmt %prec LOWER_THAN_ELSE
 96     | IF exp THEN stmt ELSE stmt
 97     ;
 98
 99 while_stmt
100     : WHILE exp DO stmt
101     ;
102
103 compound_stmt
104     : BEGINN stmts END
105     ;
106
107 exp
108     : exp PLUS exp
109     | exp MINUS exp
110     | exp TIMES exp
111     | exp DIVIDE exp
112     | exp LT exp
113     | exp LE exp
114     | exp GT exp
115     | exp GE exp
116     | exp EQ exp
117     | exp NEQ exp
118     | LPAREN exp RPAREN
119     | MINUS exp %prec UMINUS
120     | INT
121     | FLOAT
122     | ID
123     ;
124
125 %%
```

调试问题

1、在编译时出现报错，报错位置都在注释行

```
root@coder:/data/workspace/myshixun# make
gcc -c parsetest.c
bison -dv parser.y
parser.y: warning: 25 shift/reduce conflicts [-Wconflicts-sr]
gcc -c parser.tab.c
flex parser.l
parser.l:39: unrecognized rule
parser.l:39: unrecognized rule
parser.l:39: unrecognized rule
parser.l:42: unrecognized rule
parser.l:42: unrecognized rule
parser.l:42: unrecognized rule
parser.l:45: unrecognized rule
parser.l:45: unrecognized rule
parser.l:45: unrecognized rule
parser.l:51: unrecognized rule
parser.l:51: unrecognized rule
parser.l:51: unrecognized rule
parser.l:64: unrecognized rule
parser.l:64: unrecognized rule
parser.l:64: unrecognized rule
parser.l:83: unrecognized rule
parser.l:83: unrecognized rule
parser.l:83: unrecognized rule
makefile:24: recipe for target 'lex.yy.c' failed
make: *** [lex.yy.c] Error 1
root@coder:/data/workspace/myshixun#
```

```
30  id          {letter}({letter}|{digit})*
31  int_num     {digit}+
32  float_num   ({digit}+\.{digit}*)|(\.{digit}+)
33
34  /* 定义注释状态 */
35  %s COMMENT
36
37  %%
38
39  /* 换行时执行 EM_newline() 来告诉报错系统行号增加了 */
40  \n          {adjust(); EM_newline(); continue;}
41
42  /* 空白符：只更新位置，不返回任何记号 */
43  {ws}        {adjust(); continue;}
44
45  /* 注释 */
46  "(*"        {adjust(); BEGIN COMMENT;}
47  <COMMENT>"*)" {adjust(); BEGIN INITIAL;}
48  <COMMENT>.    {adjust(); }
49  <COMMENT>\n   {adjust(); EM_newline();}
50
51  /* 关键字匹配，必须在 ID 之前 */
52  "PROGRAM"   {adjust(); return PROGRAM;}
53  "VAR"       {adjust(); return VAR;}
```

👉 在 **Flex（parser.l）**文件的规则段（第二个 %% 之后），注释或代码不能顶格写。

2、反复修改代码，反复出现1，55报错

```
58
59  vardec   : VAR declist
60                       ;
61
62  declist : dec
63          | declist dec
64          ;
65
66  dec      : idlist COLON type SEMICOLON
67                       ;
68
69  idlist   : ID
70          | idlist COMMA ID
71          ;
72
```



原因：第一行的注释中存在英文`Parse Successful`，你的词法分析器把注释里的 `Parse` 或 `Successful` 当成了 `ID` 返回给语法分析器，所以语法分析器一开始没有读到 `PROGRAM`，而是读到了一个 `ID`，于是直接 `syntax error`。

👉 将`%s COMMENT` 修改为 `%x COMMENT`

`%s COMMENT` 定义的是 **包含型状态**。在 `COMMENT` 状态下，没有写状态限定的普通规则仍然会生效，所以注释中的英文单词仍然可能被识别成 `ID`。

`%x COMMENT` 是 **排他型状态**，进入注释状态后，只会匹配 `<COMMENT>` 开头的规则，普通的 `ID`、关键字、数字规则不会再生效。

实验八

调试问题

1、在命令行输入make命令后，出现如下报错：

```
root@coder:/data/workspace/myshixun# make
gcc -c parsetest.c
bison -dv parser.y
gcc -c parser.tab.c
gcc -c lex.yy.c
In file included from parser.l:8:0:
parser.y:28:9: error: unknown type name ‘a_exp’
         a_exp exp;
         ^~~~
parser.y:29:9: error: unknown type name ‘a_stm’
         a_stm stm;
         ^~~~
parser.y:30:9: error: unknown type name ‘a_stm_list’
         a_stm_list stms;
         ^~~~~~~~~~
parser.y:31:9: error: unknown type name ‘a_dec’
         a_dec dec;
         ^~~~
parser.y:32:9: error: unknown type name ‘a_dec_list’
         a_dec_list decl;
         ^~~~~~~~~~
parser.y:33:9: error: unknown type name ‘a_var_list’
         a_var_list vlist;
         ^~~~~~~~~~
parser.y:34:9: error: unknown type name ‘a_bexp’
         a_bexp bexp;
         ^~~~~~
makefile:23: recipe for target 'lex.yy.o' failed
make: *** [lex.yy.o] Error 1
```

但检查parser.y文件后发现已经包含了ast.h文件。

✨原因：虽然在 parser.y 的顶部引用了 ast.h，但由于 Bison（Yacc）的工作机制：

- **parser.tab.h 的生成**：Bison 会根据%union 定义生成一个头文件 parser.tab.h。这个文件里包含了类似 a__exp exp; 这样的代码。
- **Lex 的编译过程**：当编译 lex.yy.c（也就是词法分析器）时，会引用 parser.tab.h 以获得 Token 的定义。
- **冲突点**：如果在 lex.yy.c 中，包含 parser.tab.h **之前** 没有先包含 ast.h，编译器在处理 parser.tab.h 里的 a__exp 时，根本不知道 a__exp 是什么类型（因为 ast.h 还没被读到）。

👉 修改 **parser.l** 文件，在包含 parser.tab.h **之前**，手动包含 ast.h：



代码

ast.c

```c
/* make exp node from real */
a_exp A_RealExp(a_pos pos, double fval){
  a_exp ret = checked_malloc(sizeof(*ret));
  ret->kind = A_realExp;
  ret->pos = pos;
  ret->exp.fval = fval;
  return ret;
}

/* make exp node from id node变量表达式节点 */
a_exp A_VarExp(a_pos pos, a_id var){
  a_exp ret = checked_malloc(sizeof(*ret));
  ret->kind = A_varExp;
  ret->pos = pos;
  ret->exp.var = var;
  return ret;
}

/* make binary operands exp node二元算术运算节点 */
a_exp A_OpExp(a_pos pos, a_op op, a_exp left, a_exp right){
  a_exp ret = checked_malloc(sizeof(*ret));
  ret->kind = A_opExp;
  ret->pos = pos;
  ret->exp.biopExp.op = op;
  ret->exp.biopExp.left = left;
  ret->exp.biopExp.right = right;
  return ret;
}

/* make boolean exp node */
a_bexp A_BExp(a_pos pos, a_bop bop, a_exp left, a_exp right){
  a_bexp ret = checked_malloc(sizeof(*ret));
  //布尔表达式无需设置类型
  ret->pos = pos;
  ret->bexp.bop = bop;
```

```
  ret->bexp.left = left;
  ret->bexp.right = right;
  return ret;
}

/* make assign statement赋值语句节点 */
a_stm A_Assign (a_pos pos, a_id var, a_exp exp){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_assign;
  ret->pos = pos;
  ret->stm.assign.var = var;
  ret->stm.assign.exp = exp;
  return ret;
}

/* make if statement IF 语句节点  */
a_stm A_If(a_pos pos, a_bexp b, a_stm s1, a_stm s2){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_if;
  ret->pos = pos;
  ret->stm.iff.b = b;
  ret->stm.iff.s1 = s1;
  ret->stm.iff.s2 = s2;
  return ret;
}

/* make while statement WHILE 语句节点*/
a_stm A_While(a_pos pos, a_bexp b, a_stm s){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_while;
  ret->pos = pos;
  ret->stm.whilee.b = b;
  ret->stm.whilee.s = s;
  return ret;
}

/* make sequence statement 顺序语句（复合语句）节点*/
a_stm A_Seq(a_pos pos, a_stm_list sl){
  a_stm ret = checked_malloc(sizeof(*ret));
  ret->kind = A_seq;
  ret->pos = pos;
  ret->stm.seq = sl;
  return ret;
}
```

parser.l

沿用实验七

parser.y

```
%{
#include <stdio.h>
#include "ast.h" //该文件定义了抽象语法树（分析树）的数据结构
#include "errormsg.h"
#include "util.h"
#define YYDEBUG 1
/* 允许跟踪错误，与%debug功能相同 */

int yylex(void); /* function prototype */

/* 该函数显示错误信息s，显示时包含了错误发生的位置。*/
void yyerror(char *s)
{
 EM_error(EM_tokPos, "%s", s);
}

/* 存放抽象语法树中 "程序" 数据结构的变量，保存最终生成的语法树 */
a_prog program = NULL;

%}


 /* 定义属性值栈的类型，最后一个实验可能需要修改此处  */
%union {
        int ival;
    double fval;
    string sval;
        a_exp exp;
        a_stm stm;
        a_stm_list stms;
        a_dec dec;
        a_dec_list decl;
        a_var_list vlist;
        a_bexp bexp;
}

 /* 定义各个终结符，以及他们的属性值的类型，最后一个实验可能需要修改此处 */
%token <sval> ID /* id */
%token <ival> INT  /*整型数*/
%token <fval> FLOAT /*浮点数*/
%token INTEGER REAL  /*两种类型名：整型、实型*/
%token
```

```
  COMMA COLON SEMICOLON LPAREN RPAREN PERIOD /* 符号 , : ; ( ) . */
  PROGRAM BEGINN END VAR IF WHILE DO   /* 关键字：PROGRAM BEGIN END VAR IF WHILE Do */
  /* 想想为什么不声明为BEGIN？ */
  THEN ELSE /* 关键字：THEN ELSE */
  ASSIGN EQ NEQ LT LE GT GE /* 符号 :=     =  <>  <  <=  >  >= */
  PLUS MINUS TIMES DIVIDE /* 符号 + = * / */
%start program

  /* 定义各个非终结符的属性值类型，指定非终结符返回的类型  */
%type <decl> vardec declist
%type <dec> dec
%type <vlist> idlist
%type <stms> stmts stmtlist
%type <stm> stmt assignment_stmt if_stmt while_stmt compound_stmt
%type <exp> exp
%type <bexp> condition

/* 优先级处理 */
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
%left EQ NEQ LT LE GT GE
%left PLUS MINUS
%left TIMES DIVIDE

%%
 /* 文法：program--> PROGRAM ID; vardec BEGIN stmts END.
  *       vardec-->   VAR declist
  *       declist-->  dec | declist dec
  *       dec-->  idlist:INTEGER; | idlist:REAL;
  *       idlist-->   ID | idlist,ID
  *       stmts--> stmtlist | (empty)
  *       stmtlist-->   stmt | stmtlist;stmt | stmtlist;
  *       stmt-->   assignment_stmt | if_stmt | while_stmt | compound_stmt
  *       assignment_stmt-->   ID:=exp
  *       if_stmt-->   IF condition THEN stmt
  *                  | IF condition THEN stmt ELSE stmt
  *       while_stmt-->   WHILE condition DO stmt
  *       compound_stmt-->   BEGINN stmts END
  *       condition-->   exp LT exp  | exp LE exp | exp NEQ exp |
  *                    | exp GT exp | exp GE exp | exp EQ exp
  *       exp--> exp PLUS exp | exp MINUS exp | exp TIMES exp
  *            | exp DIVIDE exp |  exp MINUS exp | LPAREN exp RPAREN
  *            | INT | FLOAT | ID
  * 最后一个实验需要修改这个文法
  */
program :   PROGRAM ID SEMICOLON vardec BEGINN stmts END PERIOD
```

```
89                      {program = A_Prog(EM_tokPos, $2, $4, $6);}
90                ;
91
92  vardec  : VAR declist   {$$ = $2;}
93          | /* empty */   { $$ = NULL; }
94                ;
95  declist : dec           { $$ = A_DecList($1, NULL); }
96          | dec declist   { $$ = A_DecList($1, $2); }
97          ;
98
99  dec     : idlist COLON INTEGER SEMICOLON { $$ = A_VarDec(EM_tokPos, $1,
    T_int); }
100         | idlist COLON REAL SEMICOLON    { $$ = A_VarDec(EM_tokPos, $1,
    T_real); }
101         ;
102
103 idlist  : ID              { $$ = A_VarList(A_Id(EM_tokPos, $1), NULL); }
104         | ID COMMA idlist { $$ = A_VarList(A_Id(EM_tokPos, $1), $3); }
105         ;
106
107 stmts   : /* empty */ { $$ = NULL; }
108         | stmtlist    { $$ = $1; }
109         ;
110
111 stmtlist: stmt                    { $$ = A_StmList($1, NULL); }
112         | stmt SEMICOLON stmtlist { $$ = A_StmList($1, $3); }
113         ;
114
115 stmt    : assignment_stmt { $$ = $1; }
116         | if_stmt         { $$ = $1; }
117         | while_stmt      { $$ = $1; }
118         | compound_stmt   { $$ = $1; }
119         ;
120
121 assignment_stmt : ID ASSIGN exp { $$ = A_Assign(EM_tokPos, A_Id(EM_tokPos,
    $1), $3); }
122                 ;
123
124 if_stmt : IF condition THEN stmt %prec LOWER_THAN_ELSE    { $$ =
    A_If(EM_tokPos, $2, $4, NULL); }
125         | IF condition THEN stmt ELSE stmt                { $$ =
    A_If(EM_tokPos, $2, $4, $6); }
126         ;
127
128 while_stmt : WHILE condition DO stmt { $$ = A_While(EM_tokPos, $2, $4); }
129            ;
130
```

```
131 compound_stmt : BEGINN stmts END { $$ = A_Seq(EM_tokPos, $2); }
132               ;
133
134 condition : exp EQ exp    { $$ = A_BExp(EM_tokPos, A_eqOp, $1, $3); }
135           | exp NEQ exp   { $$ = A_BExp(EM_tokPos, A_neqOp, $1, $3); }
136           | exp LT exp    { $$ = A_BExp(EM_tokPos, A_ltOp, $1, $3); }
137           | exp LE exp    { $$ = A_BExp(EM_tokPos, A_leOp, $1, $3); }
138           | exp GT exp    { $$ = A_BExp(EM_tokPos, A_gtOp, $1, $3); }
139           | exp GE exp    { $$ = A_BExp(EM_tokPos, A_geOp, $1, $3); }
140           ;
141 exp     : INT               { $$ = A_IntExp(EM_tokPos, $1); }
142         | FLOAT             { $$ = A_RealExp(EM_tokPos, $1); }
143         | ID                { $$ = A_VarExp(EM_tokPos, A_Id(EM_tokPos, $1)); }
144         | exp PLUS exp      { $$ = A_OpExp(EM_tokPos, A_plusOp, $1, $3); }
145         | exp MINUS exp     { $$ = A_OpExp(EM_tokPos, A_minusOp, $1, $3); }
146         | exp TIMES exp     { $$ = A_OpExp(EM_tokPos, A_timesOp, $1, $3); }
147         | exp DIVIDE exp    { $$ = A_OpExp(EM_tokPos, A_divideOp, $1, $3); }
148         | LPAREN exp RPAREN { $$ = $2; }
149         ;
150
151
152
```